

DockFlow

Methodology and Evaluation

Clothing Warehouse & Delivery Management System

Sec No-13 | Team No-14

U. Sindhuja • P. Bhagya Sri • S. Rishitha Reddy

Guided by Dr. M. Gayatri | Koneru Lakshmaiah Education Foundation



How this review covers every criterion



**Novelty of Developed
Methodology**

10

MARKS

Slides 3-4

Our method and what is new
in it



**Adaptability of
Methodology**

10

MARKS

Slides 5-6

Change scenarios and trade-
offs



**Methodology
Evaluation**

10

MARKS

Slides 7-9

Framework, objectives and
results



**Dissemination &
Accessibility**

10

MARKS

Slides 10-11

Who can use it and how it is
shared



Presentation Skills

10

MARKS

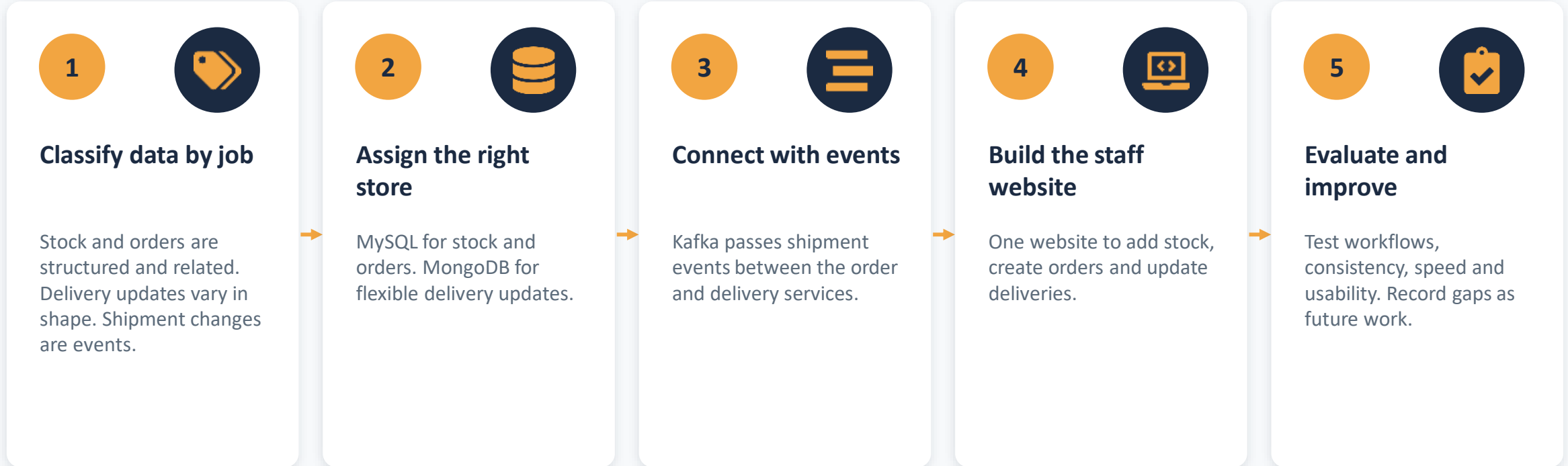
Slides 1-13

Clear flow, summary and
references

REVIEW 3 TOTAL

5 criteria x 10 marks = 50

Our methodology: choose storage by job



GUIDED BY THE LITERATURE

Pick storage by workload [6]. Use events between services [1]. Keep the cost of several databases under control [5].

What is new in our methodology

Aspect	Typical approach	DockFlow methodology
How data is stored	One database for every kind of data	Each data type goes to the store that suits its job
Delivery updates	Fixed table columns	Flexible documents in MongoDB
Service communication	Direct calls between modules	Shipment events through Kafka
Choice of tools	Chosen by habit or convenience	Chosen by workload, guided by the literature [5] [6]
Scope	Often only stock, or only delivery	Stock, orders and deliveries in one website

The novelty is the combination, applied to a clothing warehouse and its deliveries: the gap found in Review 2.

How the methodology adapts to change



New clothing category

WHAT CHANGES

Add the category and its sizes to the stock data.

WHAT STAYS

Order and delivery flow.



New delivery details

WHAT CHANGES

Add fields to the flexible delivery updates in MongoDB.

WHAT STAYS

Stock and order tables in MySQL.



Courier GPS feed (future)

WHAT CHANGES

Add a new source of shipment events through Kafka.

WHAT STAYS

Existing services and tables.



New service, such as notifications

WHAT CHANGES

Add a new service that listens to shipment events.

WHAT STAYS

Services already running.

Design reasoning for these scenarios; they have not yet been tested in DockFlow.

Trade-offs and how we manage them

Trade-off	Why it happens	How we manage it
Three tools to run	MySQL, MongoDB and Kafka each need setup and care.	Give each tool one narrow job.
Data in two stores	Orders live in MySQL and delivery updates in MongoDB.	Keep records linked by an order reference.
Repeated events	Kafka delivery is at-least-once, so an event can arrive twice [1].	Make delivery updates safe to repeat.
No GPS or payments	Out of scope for this course project.	Planned as new events and services later.

Review 2 [5] also found that polyglot persistence adds complexity. We accept this cost for a better fit per job.

Evaluation framework: five measures

Criterion	How it is tested	Measure
Functional correctness	Run each staff workflow: add stock, create an order, update a delivery	Test cases passed
Data consistency	Compare stock counts with orders; check delivery updates match their orders	Mismatches found
Performance	Time common actions on MySQL (stock, orders) and MongoDB (delivery updates)	Response time per action
Event flow	Send shipment events through Kafka and check the delivery side	Events received; repeats handled
Usability	Ask users to complete the three staff tasks	Tasks completed; time taken

Each measure checks one claim of the methodology: right store for each job, consistent data, working events and usable workflows.

Outcome against our Review 1 objectives

1

Track men's, women's and kids' clothes by size, color and location

IMPLEMENTED

2

Let staff add stock and create orders from one website

IMPLEMENTED

3

Record delivery updates and keep them easy to follow

IMPLEMENTED

4

Use the right database for each job: MySQL, MongoDB and Kafka

IMPLEMENTED



Known limits

- Does not connect to real courier GPS
- Does not handle payments
- CI/CD is not yet in place
- Full C4 diagrams still to be drawn
- Measured results are on the next slide

Measured results

Measure	Suggested target	Measured result
Workflow test cases passed	All cases pass	Add result
Stock vs order mismatches	None	Add result
Response time: stock and order actions	Under 1 second	Add result
Delivery update saved after a shipment event	Every event	Add result
Users completing the three staff tasks	All users	Add result

Results come from running the five measures on the working system; limits of testing are stated openly.

Accessibility: easy for staff to use



Browser-based

Staff open DockFlow in a web browser.



Simple workflow

Three main tasks: add stock, create orders, update deliveries.



Easy to find items

Men's, women's and kids' clothes by size, color and location.



Clear status

Delivery updates keep shipment status in view.

SUGGESTED ACCESSIBILITY IMPROVEMENTS

Readable colors and text size

Keyboard navigation

Mobile-friendly layout

Clear error messages

Dissemination: how we share the work

Audience	What they get	Channel	Status
Faculty reviewers	Working demo and review slides	Review sessions	Ready
Classmates	A walkthrough of the workflows	Class demo	Planned
Future developers	Source code and a setup guide	Code repository with README	Planned
Wider audience	A short write-up of the method	Project report or poster	Planned

Full C4 diagrams, one of our planned improvements, will make the design easier to explain, share and hand over.

Summary: DockFlow Review 3



Novel

Data classified by job:
MySQL, MongoDB and Kafka.



Adaptable

New categories, fields and
services fit with local change.



Evaluated

Five measures; objectives
checked; limits stated.



Accessible

Browser-based, three-task
workflow, sharing plan.

Thank you

We are happy to take your questions.

U. Sindhuja • P. Bhagya Sri • S. Rishitha Reddy
Team No-14 | Guided by Dr. M. Gayatri

References

- [1] J. Kreps, N. Narkhede and J. Rao, "Kafka: a distributed messaging system for log processing," NetDB'11, Athens, Greece, 2011.
- [5] F. Halili, A. Nuhiji and D. Mustafai-Veliu, "Polyglot Persistence in Microservices: Managing Data Diversity in Distributed Systems," arXiv:2509.08014, 2025.
- [6] "Benchmarking SQL and NoSQL Persistence in Microservices Under Variable Workloads," Future Internet, vol. 18, art. 53, 2026.

Numbering follows the reference list in Review 2.